

Note 3: The dockerfile-maven-plugin and Pushing to Registries

What is the dockerfile-maven-plugin?

The dockerfile-maven-plugin by Spotify is a Maven plugin that integrates Docker image building and pushing directly into the Maven lifecycle. It works with an existing Dockerfile in your project and binds Docker build/push operations to Maven lifecycle phases.

Adding the Plugin to pom.xml

```
<build>

  <plugins>

    <plugin>

      <groupId>com.spotify</groupId>

      <artifactId>dockerfile-maven-plugin</artifactId>

      <version>1.4.13</version>

    </plugin>

  </plugins>

  <executions>

    <!-- Bind docker:build to the package phase -->

    <execution>

      <id>build-image</id>

      <phase>package</phase>

      <goals>

        <goal>build</goal>

      </goals>

    </execution>

    <!-- Bind docker:push to the deploy phase -->

    <execution>

      <id>push-image</id>

      <phase>deploy</phase>

      <goals>

        <goal>push</goal>

      </goals>

    </execution>

  </executions>

</build>
```

```

        </goals>

    </execution>

</executions>

<configuration>

    <!-- Docker Hub username/repository name -->

    <repository>myusername/my-app</repository>

    <!-- Tag the image with the Maven project version -->

    <tag>${project.version}</tag>

    <!-- Build arguments passed into the Dockerfile -->

    <buildArgs>

        <JAR_FILE>target/${project.build.finalName}.jar</JAR_FILE>

    </buildArgs>

</configuration>

</plugin>

</plugins>

</build>

```

How it Works with Lifecycle Phases

With the above configuration:

- `mvn package` → compiles, tests, packages the JAR, then builds the Docker image
- `mvn deploy` → runs all of the above, then pushes the image to Docker Hub

The image is automatically tagged with the Maven project version. If your POM has `<version>1.2.0</version>`, the image is tagged as `myusername/my-app:1.2.0`. If it is a SNAPSHOT version like `1.2.0-SNAPSHOT`, the tag becomes `myusername/my-app:1.2.0-SNAPSHOT`.

Dockerfile Used with the Plugin

When using the dockerfile-maven-plugin, the Dockerfile typically expects a pre-built JAR to be copied in — Maven handles the build, Docker handles the image:

```
FROM eclipse-temurin:17-jre-alpine
```

```
WORKDIR /app
```

```
ARG JAR_FILE
```

```
COPY ${JAR_FILE} app.jar
```

```
EXPOSE 8080
```

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

The ARG JAR_FILE receives the value passed via <buildArgs> in the plugin configuration, making it flexible across projects.

Authentication for Docker Registries

Docker Hub Authentication

Before pushing to Docker Hub, you must authenticate. The plugin uses Docker's credential store. Log in once from the command line:

```
docker login
```

This stores credentials in ~/.docker/config.json, which the plugin reads automatically.

Storing Credentials in Maven settings.xml

For CI/CD pipelines where docker login is not interactive, credentials can be stored in Maven's settings.xml located at ~/.m2/settings.xml:

```
<settings>
```

```
  <servers>
```

```
    <server>
```

```
<id>docker.io</id>

<username>myusername</username>

<password>mypassword</password>

</server>

</servers>

</settings>
```

The `<id>` must match the registry hostname. For Docker Hub it is `docker.io`, for GitHub Container Registry it is `ghcr.io`, for AWS ECR it follows the format `<account-id>.dkr.ecr.<region>.amazonaws.com`.

Important: Never commit `settings.xml` with real credentials to source control. In CI/CD pipelines, credentials are injected as environment variables or secrets.

Pushing to Different Registries

Pushing to Docker Hub

```
<configuration>

  <repository>docker.io/myusername/my-app</repository>

  <tag>${project.version}</tag>

</configuration>
```

Run:

```
mvn deploy
```

Pushing to GitHub Container Registry (GHCR)

```
<configuration>

  <repository>ghcr.io/myusername/my-app</repository>

  <tag>${project.version}</tag>

</configuration>
```

Authenticate with a GitHub Personal Access Token (PAT):

```
echo $GITHUB_TOKEN | docker login ghcr.io -u myusername --password-stdin
```

Pushing to a Private Registry (e.g., Nexus or Artifactory)

```
<configuration>
```

```
<repository>registry.mycompany.com/myteam/my-app</repository>

<tag>${project.version}</tag>

</configuration>
```

Tagging Strategy

Proper image tagging is critical for traceability in CI/CD pipelines. Common tagging strategies:

Tag	When to Use
-----	-------------

<code>\${project.version}</code>	Always — ties image to Maven version
----------------------------------	--------------------------------------

<code>latest</code>	For the most recent stable build
---------------------	----------------------------------

Git commit SHA	For precise traceability in pipelines
----------------	---------------------------------------

<code>\${project.version}-\${buildNumber}</code>	For unique CI build identification
--	------------------------------------

You can push multiple tags by running the build and deploy goals multiple times with different tag configurations, or by using a more feature-rich plugin like `docker-maven-plugin` (fabric8) which supports multiple tags natively.

The fabric8 docker-maven-plugin (Brief Overview)

The fabric8 plugin is more powerful than the Spotify plugin and does not require a separate Dockerfile — the image configuration is defined entirely inside the POM:

```
<plugin>

  <groupId>io.fabric8</groupId>

  <artifactId>docker-maven-plugin</artifactId>

  <version>0.43.4</version>

  <configuration>

    <images>

      <image>

        <name>myusername/my-app:${project.version}</name>

        <build>

          <from>eclipse-temurin:17-jre-alpine</from>
```

```

    <assembly>
      <descriptorRef>artifact</descriptorRef>
    </assembly>
    <cmd>
      <shell>java -jar /maven/${project.build.finalName}.jar</shell>
    </cmd>
    <ports>
      <port>8080</port>
    </ports>
  </build>
</image>
</images>
</configuration>
</plugin>

```

Goals:

- mvn docker:build — builds the image
- mvn docker:push — pushes to registry
- mvn docker:run — runs the container
- mvn docker:stop — stops the container

Complete Maven + Docker Workflow Summary

Step	Command	What Happens
Compile	mvn compile	Java source compiled to .class files
Test	mvn test	Unit tests run via Surefire
Package	mvn package	JAR created; Docker image built
Install	mvn install	JAR installed to local .m2 repo
Deploy	mvn deploy	JAR pushed to Maven registry; Docker image pushed to container registry

Clean build mvn clean package Fresh JAR + fresh Docker image